

Compilador fase 2: Análise Semântica e Geração de Código¹

Na **fase 1** do projeto, foi implementado um compilador que fazia a leitura de um **arquivo-fonte** na linguagem **TINY-C** e realizava a análise léxica e sintática do programa fonte. O objetivo dessa fase é implementar as fases de **Análise Semântica** e **Geração de Código Intermediário** do compilador, a implementação dessa fase será baseada na implementação realizada na **fase 1**, caso você não tenha implementado a **fase 1**, para essa fase, você **terá que implementar tanto a fase 1 quanto a fase 2**.

Análise Semântica

Na **Análise Semântica** o seu compilador deverá verificar se as construções sintáticas da fase anterior estão coerentes, o compilador implementado na fase anterior deve manter as funcionalidades de identificação de erros **léxicos** e **sintáticos**, e adicionalmente, emitir as mensagens de **erros semânticos**, caso ocorram.

Basicamente o Compilador fará somente a verificação semântica para variáveis em dois momentos:

- **Declaração:** Na seção de declaração de variável `<var_decl_list>` o compilador deve garantir que os identificadores usados no nome de variável sejam únicos, ou seja, não podendo ter duas variáveis declaradas com o mesmo identificador. Caso aconteça uma repetição de identificador o **compilador deve ser finalizado informando que ocorreu um erro semântico** explicativo. Para isso deverá ser implementado uma **minitabela de símbolos** que armazenará as variáveis declaradas (**identificador** e **endereço**), o endereço da variável seria a ordem em que a variável foi declarada, dessa forma a primeira variável tem endereço **0**, a segunda endereço **1** e assim sucessivamente.
- **Corpo do programa:** As variáveis declaradas na seção de declaração de variáveis podem ser referenciadas nos comandos de atribuição, nas expressões e nas chamadas das funções de entrada e saída. Portanto toda vez que uma variável for referenciada no corpo de programa, o compilador deve verificar se a variável foi declarada corretamente, **caso não tenha sido declarada é gerado um erro semântico** explicativo e compilador é finalizado.

Para armazenar as informações referentes as variáveis, a **minitabela de símbolos** deverá, **obrigatoriamente**, ser implementada utilizando a estrutura de dados **lista encadeada**, cada nó da tabela de símbolos é do tipo **TNo** descrito abaixo:

```
typedef struct _TNo{
    char ID[16];
    int endereco;
    struct _TNo *prox;
}TNo;
```

Para simplificar a análise semântica e geração de código intermediário, o compilador não precisará realizar a distinção entre expressões inteiras e lógicas, assim, para os testes do compilador, só teremos variáveis e expressões do tipo inteiro (**int**), também, não teremos construções que misturam os dois tipos, por exemplo: **25+(x>y)**.

Geração de Código Intermediário

A **Geração de Código Intermediário** será baseada na proposta do livro do professor **Tomasz Kowaltowski** Implementação de Linguagem de Programação (**Seção 10.3 Análise Sintática e Geração de Código**). A geração das instruções da MEPA será realizada nas funções mutuamente recursivas implementadas no analisador sintático, para tanto basta **imprimir as instruções da MEPA** nas mesmas

¹ **Importante:** A especificação desse trabalho pode sofrer modificações de acordo com discussões que tivermos em sala de aula.

funções que fazem análise sintática e semântica do compilador. Nos exemplos de geração de código intermediário são apresentadas duas funções: **proximo_rotulo()** e **busca_tabela_simbolos()**, recomendamos fortemente que você implemente conforme proposto, pois tornam a implementação modularizada, a geração de código mais simples e poderão ser utilizadas questões da prova da disciplina.

Como primeiro exemplo considere a produção abaixo para o comando condicional **<cond_stmt>**, conforme visto na gramática da **fase 1 do compilador**.

<cond_stmt> ::= if '(' <expr> ')' <stmt> [else <stmt>]

No quadro segue a implementação da função correspondente a produção que gera código intermediário:

```
void cond_stmt(){
    int L1 = proximo_rotulo();
    int L2 = proximo_rotulo();
    consome(IF);
    consome(ABRE_PAR);
    expr();
    consome(FACHA_PAR);
    printf("\tDSVF L%d\n",L1);
    stmt();
    printf("\tDSVS L%d\n",L2);
    printf("L%d:\tNADA\n",L1);
    if(lookahead == ELSE){
        consome(ELSE);
        stmt();
    }
    printf("L%d:\tNADA\n",L2);
}
```

A função **proximo_rotulo()** retorna o valor do próximo rótulo consecutivo positivo (por exemplo **1, 2, 3, ...**). **Importante:** Como todas as funções são recursivas, deve-se tomar o cuidado na atribuição das variáveis locais que irão receber o retorno da função e/ou a ordem de chamadas da função **proximo_rotulo()**.

Como explicado acima, só vamos considerar variáveis do tipo **int**, logo a produção **<factor>** só precisa gerar código para **id**, **intconst** e expressão entre parênteses, como abaixo:

<factor> ::= intconst | id | '(' <expr> ')'

No quadro é apresentado o segundo exemplo de implementação de geração de código intermediário:

```
void factor(){
    if(lookahead == ID ){
        int endereco = busca_tabela_simbolos(InfoAtomo.atributo_ID);
        printf("\tCRVL %d\n",endereco);
        consome(lookahead);
    }
    else if(lookahead == INTCONST){
        printf("\tCRCT %d\n", InfoAtomo.atributo_numero);
        consome(lookahead);
    }else{
        consome(ABRE_PAR);
        expr();
        consome(FECHA_PAR);
    }
}
```

Nesse exemplo utilizamos a função **busca_tabela_simbolos()**, que recebe como parâmetro o atributo **atributo_ID** do átomo corrente (um vetor de caracteres) e retorna o endereço da variável armazenado na minitabela de símbolos, caso o identificador **não conste da tabela de símbolos a função gera um erro semântico**, que é informado na tela do computador, em seguida o processo de compilação é finalizado.

Lembre-se que a variável **InfoAtomo** é uma variável global do tipo **TInfoAtomo** e é atualizada na função **consome()** e armazena os atributos do **átomo** reconhecido no **analisador léxico**. Sugestão: declare também o início da lista encadeada em uma variável global de forma a facilitar a implementação.

Execução do Compilador – fase 2

A seguir temos um programa em **TINY-C** que calcula o fatorial de um número lido no início do programa, considere que o programa **fatorial** não possui erros léxicos e sintáticos.

```
1  /*
2  programa calcula o fatorial de um numero lido
3  */
4  void main ( void ) {
5      int fat,num,cont;
6      readint(num);
7      fat = 0x1;
8      cont = 0x2;
9      while( cont <= num ){
10         fat = fat * cont;
11         cont = cont + 0x1;
12     }
13     writeint(fat) // imprime o fatorial calculado
14 }
```

Na saída do **compilador**, caso o programa não tenha erro léxico sintático ou semântica, deve ser impresso, no monitor do computador, a **tradução do código fonte para linguagem MEPA e a tabela de símbolos com as variáveis declaradas com seus endereços**.

Se o código fonte tiver erros, deve ser gerada uma mensagem informativa imprimindo o número da linha no arquivo fonte, onde o primeiro erro ocorre, e qual o erro foi identificado. **Não é necessário imprimir os átomos reconhecidos na fase léxica.**

Saída do compilador:

```
INPP
AMEM 3
LEIT
ARMZ 1
CRCT 1
ARMZ 0
CRCT 2
ARMZ 2
L1: NADA
CRVL 2
CRVL 1
CMEG
DVSF L2
CRVL 0
CRVL 2
MULT
ARMZ 0
CRVL 2
CRCT 1
```

SOMA
ARMZ 2
DSVS L1
L2: NADA
CRVL 0
IMPR
PARA

TABELA DE SIMBOLOS

fat		Endereco: 0
num		Endereco: 1
cont		Endereco: 2

Observações importantes:

O programa deve estar bem documentado e pode ser feito em grupo de até **2 alunos**, não esqueçam de colocar o **nome dos integrantes** do grupo no arquivo fonte do trabalho e sigam as **Orientações para Desenvolvimento de Trabalhos Práticos** disponível no **Moodle**.

Fica **terminantemente proibido** o uso de ferramentas de **Inteligência Artificial**, como o ChatGPT, para a **geração automática do código do projeto**. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o **COMUNICADO DA FCI de 05/02/2025**.

O trabalho será avaliado de acordo com os seguintes critérios:

- Funcionamento do programa, caso programa apresentarem **warning** ao serem compilados serão penalizados. Após a execução o programa deve finalizar com **retorno igual a 0**;
- Caso o **programa não compile ou não execute** será penalizado com a **NOTA 0.0**;
- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador do **MinGW** com **VSCode**, para configurar sua máquina no Windows acesse:
<https://www.doug.dev.br/2022/Instalacoes-e-configuracoes-para-programar-em-C-usando-o-VS-Code/>
- Compile seu programa com o seguinte comando abaixo, considere que o programa fonte do seu compilador seja compilador.c:
`gcc -g -Og -Wall compilador.c -o compilador`
- O quão fiel é o programa quanto à descrição do enunciado, principalmente ao formato de do **arquivo de entrada**;
- Clareza e organização, programas com código confuso (linhas longas, variáveis com nomes não-significativos,) e desorganizado (sem indentação, sem comentários,) também serão levados em consideração na correção.
- Entrega de um arquivo **Readme.txt** explicando até a parte do trabalho que foi feito.
- Para esclarecer alguma dúvida quanto a entrega do projeto, poderá ser **requisitado uma entrevista com os integrantes** que implementaram o projeto, com intuito de validar se as regras acima foram todas atendidas.